

Prática 5: Processamento em GPU com Python

Lucas Arruk Mendes¹

Pedro Augusto Faria²

¹ RA: 819446, Departamento de Computação, Universidade Federal de São Carlos

² RA: 821124, Departamento de Computação, Universidade Federal de São Carlos

Resumo:

Esta prática avaliou o uso de GPU em uma FFT e no cálculo do conjunto de Mandelbrot. As versões originalmente propostas em MATLAB foram implementadas em Python com NumPy, CuPy e Numba CUDA. Foram medidos os tempos de execução, os custos de transferência e os respectivos speedups. A FFT atingiu speedup de 69,875 sem transferência e 15,259 incluindo a transferência para a GPU. No Mandelbrot, o kernel CUDA atingiu speedup total de 45,079 e produziu resultado idêntico às demais implementações.

Palavras-chave: GPU; CUDA; FFT; Mandelbrot; Python.

1 Introdução

O objetivo da prática foi comparar processamento em CPU e GPU em dois problemas: a transformada rápida de Fourier (FFT) e o conjunto de Mandelbrot. Foi autorizado o uso de Python em substituição ao MATLAB. Assim, NumPy foi usado na CPU, CuPy representou operações vetorizadas sobre dados residentes na GPU e Numba CUDA foi usado para implementar um kernel explícito.

Os experimentos foram executados em uma NVIDIA GeForce RTX 2060 com 6 GB de memória. Cada tempo apresentado corresponde à mediana de cinco execuções.

2 Fundamentos

O speedup foi calculado por

$$S = \frac{T_{\text{CPU}}}{T_{\text{GPU}}}, \quad (1)$$

considerando separadamente o tempo de computação e o tempo que inclui a transferência de dados.

Na FFT, um `numpy.ndarray` permanece na memória principal, enquanto um `cupy.ndarray` permanece na memória da GPU. A conversão com `cp.asarray` transfere dados para o dispositivo e `cp.asnumpy` realiza a transferência inversa (CuPy Developers, 2026).

Para o Mandelbrot foi utilizada a relação

$$Z_{n+1} = Z_n^2 + c, \quad Z_0 = c. \quad (2)$$

Um ponto continua sendo iterado enquanto $|Z_n| \leq 2$. O número de iterações antes do escape define o valor do pixel correspondente no plano da imagem (Wikipedia Contributors, 2026).

3 Métodos

3.1 Exercício 1: FFT

Foi criada uma matriz aleatória `float32` de dimensão 3000×3000 . A FFT foi aplicada sobre a primeira dimensão, reproduzindo o comportamento do exemplo do enunciado. Foram medidos:

1. FFT em CPU com NumPy;
2. somente a FFT em GPU;
3. transferência host-device seguida da FFT;
4. custo médio por FFT ao reutilizar dados residentes na GPU.

Eventos CUDA e sincronização explícita foram usados para impedir que a execução assíncrona produzisse tempos incorretos. Os resultados CPU e GPU foram comparados com tolerância numérica.

3.2 Exercício 2a: CPU

O plano complexo $[-2, 1] \times [-1,5, 1,5]$ foi mapeado para uma matriz 1000×1000 . Para cada pixel, o algoritmo:

1. calcula o número complexo c correspondente;
2. inicializa $Z_0 = c$;
3. aplica a recorrência até 500 vezes;
4. interrompe quando $|Z| > 2$;
5. grava na matriz a última iteração válida.

A versão serial foi compilada com `numba.njit`.

3.3 Exercício 2b: GPU vetorizada

A alternativa ao `gpuArray` do MATLAB foi implementada com matrizes CuPy. A cada iteração, operações vetorizadas atualizam todos os pontos ativos da matriz simultaneamente.

3.4 Exercício 2c: kernel CUDA

A alternativa ao `arrayfun` foi um kernel Numba CUDA com um thread por pixel (Numba Developers, 2026). Foram usados blocos de 16×16 threads. Todas as versões utilizaram precisão `float32`, garantindo uma comparação direta das matrizes resultantes.

4 Resultados

4.1 FFT

A Tabela 1 apresenta os tempos e speedups. A inclusão da transferência reduziu o ganho, mas a GPU permaneceu mais rápida que a CPU.

Tabela 1 — Resultados da FFT para matriz 3000×3000 .

Caso	Tempo (ms)	Speedup
CPU	137,556	1,000
GPU	1,969	69,875
GPU + transferência	9,015	15,259
GPU residente	2,953	46,582

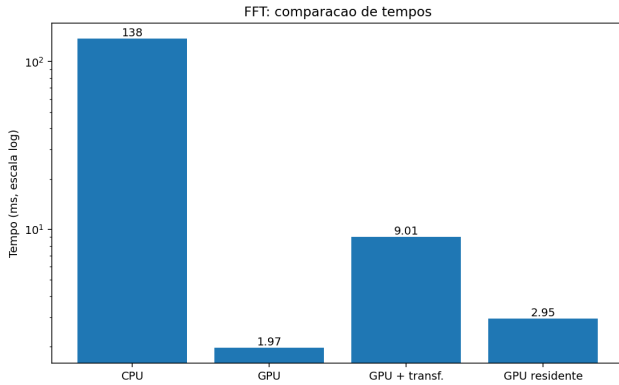


Figura 1. Tempos medidos para a FFT.

As saídas foram numericamente equivalentes, com diferença absoluta máxima de $1,85 \times 10^{-4}$.

4.2 Mandelbrot

A Tabela 2 mostra os resultados para grid 1000×1000 e 500 iterações.

Tabela 2 — Resultados do conjunto de Mandelbrot.

Implementação	Tempo (ms)	Speedup
CPU serial	210,842	1,000
GPU vetorizada	230,551	0,915
Kernel CUDA	4,677	45,079

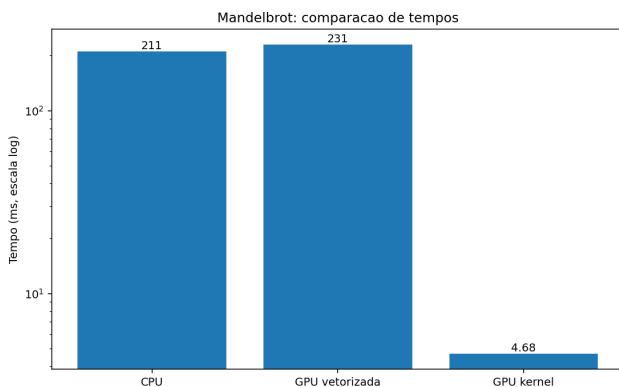


Figura 2. Comparação dos tempos do Mandelbrot.

As matrizes produzidas pelas três implementações foram exatamente iguais: nenhum pixel apresentou diferença.

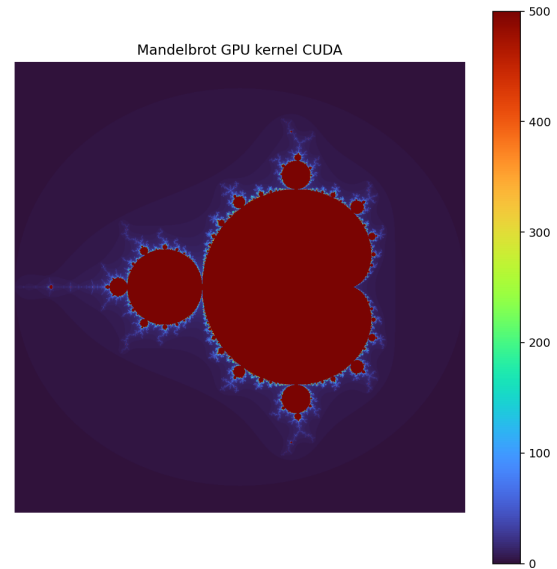


Figura 3. Conjunto de Mandelbrot obtido pelo kernel CUDA.

5 Discussão

No Exercício 1, o speedup inicial foi 69,875. Ao incluir a transferência host–device, ele caiu para 15,259, pois o custo de movimentação pela interface PCIe passou a fazer parte do tempo total. A solução pesquisada foi manter os dados como `cupy.ndarray` e executar várias operações antes de retorná-los à CPU. Com a transferência amortizada, o speedup foi 46,582.

No Exercício 2b, a versão vetorizada foi ligeiramente mais lenta que a CPU. Embora execute na GPU, ela cria matrizes temporárias em cada uma das 500 iterações e realiza várias operações globais. Já o Exercício 2c concentra todo o laço em um kernel e atribui um pixel a cada thread, reduzindo lançamentos e tráfego de memória. Por isso, alcançou speedup total de 45,079.

A igualdade exata entre as matrizes confirmou que a paralelização preservou o resultado da implementação serial.

6 Conclusão

Todos os itens da prática foram implementados em Python. A FFT demonstrou que o custo de transferência reduz significativamente o speedup e que manter os dados na GPU recupera parte desse ganho. No Mandelbrot, a forma de paralelização foi determinante: a implementação vetorizada não superou a CPU, enquanto o kernel CUDA foi 45,079 vezes mais rápido e manteve resultado idêntico ao caso serial.

Referências

- CuPy Developers (2026). *CuPy Documentation*. URL: <https://docs.cupy.dev/> (acesso em 10/06/2026).
- Numba Developers (2026). *CUDA Kernel API*. URL: <https://numba.readthedocs.io/en/stable/cuda-reference/kernel.html> (acesso em 10/06/2026).
- Wikipedia Contributors (2026). *Mandelbrot set*. URL: https://en.wikipedia.org/wiki/Mandelbrot_set (acesso em 10/06/2026).